1 Welcome to the cloud, Spring

This chapter covers

- Understanding microservices architectures
- Understanding why companies use microservices
- Using Spring, Spring Boot, and Spring Cloud for building microservices
- Understanding the cloud and cloud-based computing models

Implementing any new architecture is not an easy task; it comes with many challenges such as application scalability, service discovery, monitoring, distributed tracing, security, management, and more. This book will introduce you to the world of microservices in Spring, teach you how to tackle all those challenges, and show you the trade-offs to consider when considering microservices for your business applications. As you go, you'll learn how to build microservice applications using technologies such as Spring Cloud, Spring Boot, Swagger, Docker, Kubernetes, ELK (Elasticsearch, Logstash, and Kibana), Stack, Grafana, Prometheus, and more.

If you are a Java developer, this book will provide a smooth migration path from building traditional Spring applications to microservice applications that can be deployed to the cloud. This book uses practical examples, diagrams, and descriptive texts to provide further details of how to implement microservice architectures.

In the end, you will have learned how to implement technologies and techniques such as client load balancing, dynamic scaling, distributed tracing, and more, to create flexible, modern, and autonomous microservice-based business applications with Spring Boot and Spring Cloud. You will also be able to create your own build/deployment pipelines to achieve continuous delivery and integration with your business by applying technologies such as Kubernetes, Jenkins, and Docker.

1.1 The evolution towards a microservices architecture

Software architecture refers to all the fundamental parts that establish the structure, operation, and interaction between the software components. This book explains how to create a microservice architecture that consists of loosely coupled software services that

carry out a small number of well-defined tasks and communicate using messages over a network. Let's start by considering the differences between microservices and some other common architectures.

1.1 N-tier architecture

One common type of enterprise architecture is the multi-layered or n-tier architecture. With this design, an applications is divided into multiple layers, each with their own responsibilities and functions, such as UI, services, data, testing, and so forth. For example, as you create your application, you make a specific project or solution for the UI, then another one for the services, another for the data layer, and so on. In the end, you will have several projects that, combined, create an entire application. For large enterprise systems, n-tier applications have many advantages, including these:

- N-tier applications offer good separation of concerns, making it possible to consider areas like UI (user interface), data, and business logic separately.
- It's easy for teams to work independently on different components of n-tier applications.
- Because this is a well-understood enterprise architecture, it's relatively easy to find skilled developers for n-tier projects.

N-tier applications also have drawbacks, such as these:

- You must stop and restart the entire application when you want to make a change.
- Messages tend to pass up and down through the layers, which can be inefficient.
- Once it's deployed, refactoring a large n-tier application can be difficult.

Although some of the topics we'll discuss in this book relate directly to n-tier applications, we will focus more directly on distinguishing microservices from another common architecture, often called the monolith.

1.2 What's a monolithic architecture?

Many small-to-medium web-based applications are built using a monolithic architectural style. In a monolithic architecture, an application is delivered as a single deployable software artifact. All of the UI, business, and database access logic are packaged together into a unique application and deployed to an application server. Figure 1.1 shows the basic architecture of this application.

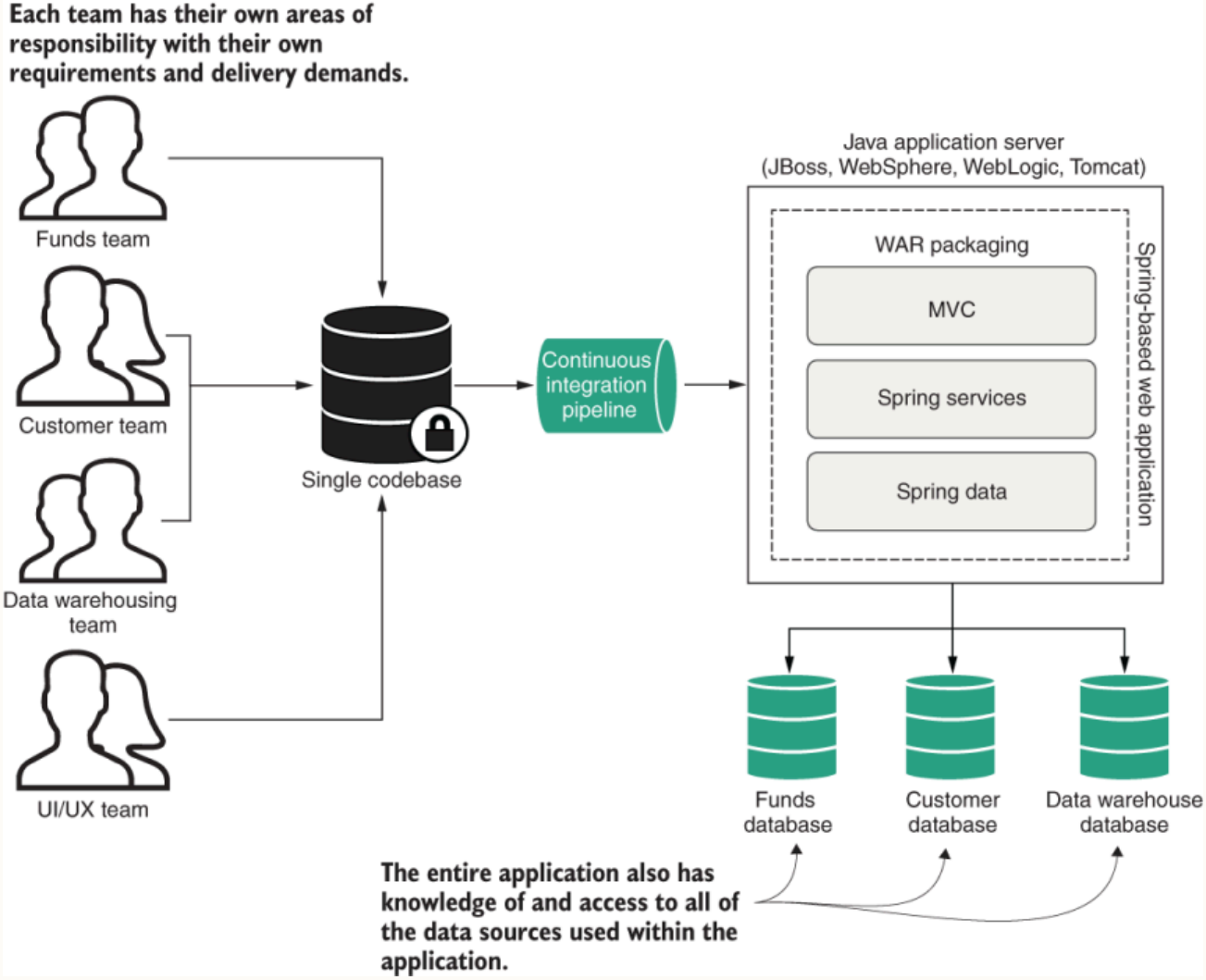


Figure 1.1 Monolithic applications force multiple development teams to synchronize their delivery date because their code needs to be built, tested, and deployed as an entire unit.

While an application might be deployed as a single unit of work, often there are multiple development teams working on a single application. Each development team is responsible for their own discrete piece of the application that usually targets specific customers. For example, imagine a scenario where we have an in-house, custom-built customer relations management (CRM) application that involves the coordination of multiple teams, including UI/UX, customer, data warehouse, and financial players, or more.

Although monolithic applications are sometimes described in negative terms by proponents of microservices architecture, these are often a great choice. Monoliths are easier to build and deploy than more complex architectures like n-tier or microservices. If your use case is well defined and unlikely to change, it can be a good decision to start with a monolith.

When an application begins to increase in size and complexity, however, monoliths can become difficult to manage. Each change to a monolith can have a cascading effect on other parts of the application, which may make it time consuming and expensive, espe-

cially in a production system. Our third option, the microservices architecture, offers the potential of greater flexibility and maintainability.

1.3 What's a microservice?

The concept of a microservice initially crept into the software development community's consciousness as a direct response to many of the challenges of trying to scale (both technically and organizationally) large monolithic applications. A *microservice* is a small, loosely coupled, distributed service. Microservices let you take an extensive application and decompose it into easy-to-manage components with narrowly defined responsibilities. Microservices help combat the traditional problems of complexity in a large codebase by decomposing it down into small, well-defined pieces.

The key concepts you need to embrace as you think about microservices are *decomposing* and *unbundling*. The functionality of your applications should be entirely independent of one another. If we take the CRM application mentioned previously and decompose it into microservices, it might look something like figure 1.2.

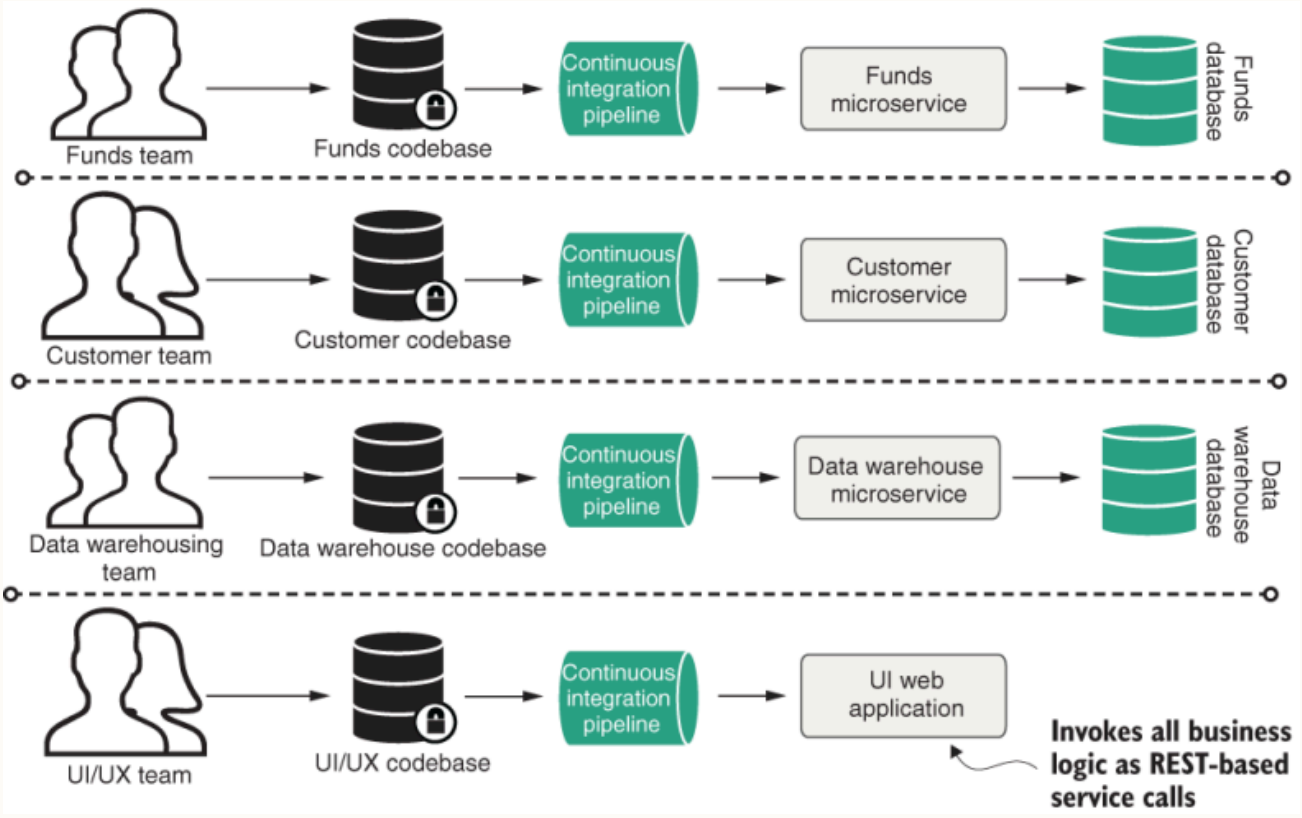


Figure 1.2 Using a microservice architecture, a CRM application is decomposed into a set of completely independent microservices, allowing each development team to move at its own pace.

Figure 1.2 shows how each team completely owns their service code and service infrastructure. They can build, deploy, and test independently of each other because their code, source control repository, and infrastructure (app server and database) are now

entirely independent of the different parts of the application. To recap, a microservice architecture has the following characteristics:

- Application logic is broken down into small-grained components with well-defined, coordinate boundaries of responsibility.
- Each component has a small domain of responsibility and is deployed independently of the others. A single microservice is responsible for one part of a business domain.
- Microservices employ lightweight communication protocols such as HTTP and JSON (JavaScript Object Notation) for exchanging data between the service consumer and service provider.
- Because microservice applications always communicate with a technology-neutral format (JSON is the most common), the underlying technical implementation of the service is irrelevant. This means that an application built using a microservice approach can be constructed with multiple languages and technologies.
- Microservices—by their small, independent, and distributed nature—allow organizations to have smaller development teams with well-defined areas of responsibility. These teams might work toward a single goal, such as delivering an application, but each team is responsible only for the services on which they're working.

Figure 1.3 compares a monolithic design with a microservices approach for a typical small e-commerce application.

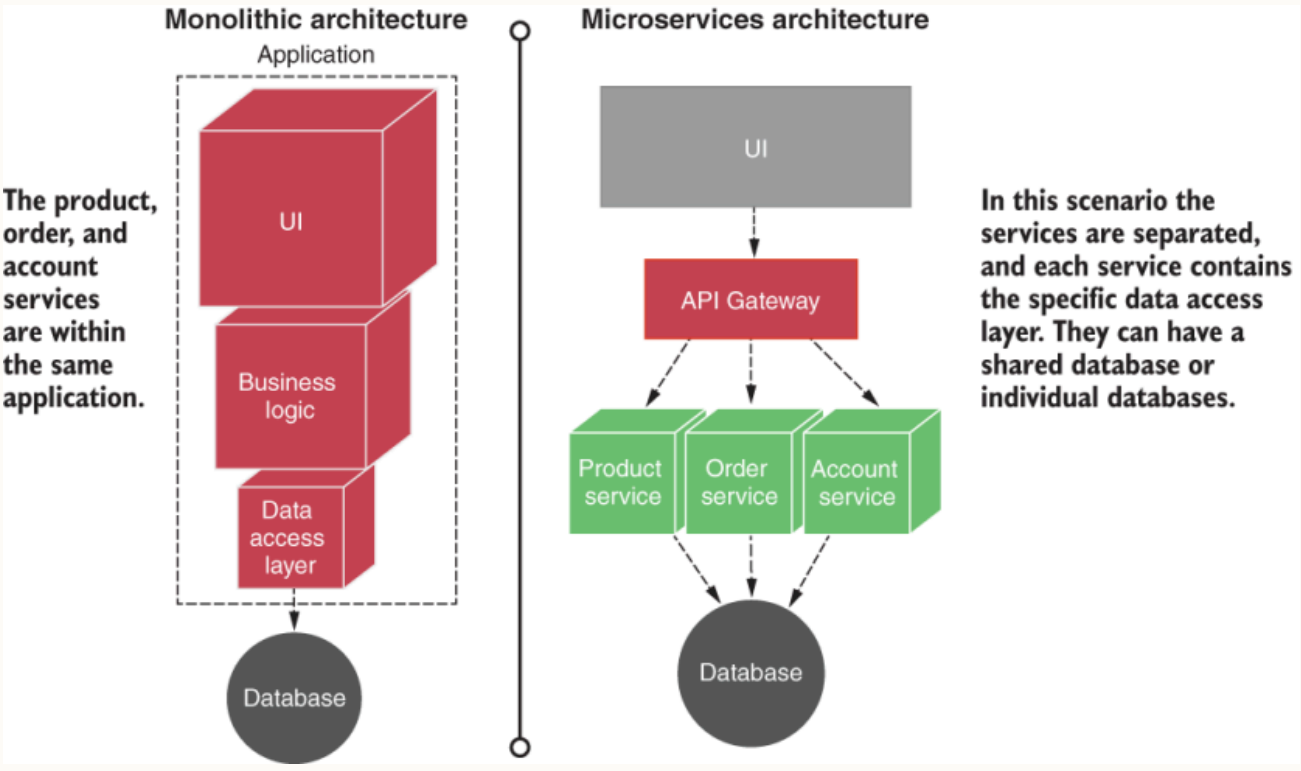


Figure 1.3 Comparing monolithic and microservices architectures

1.4 Why change the way we build applications?

Companies that used to serve local markets are suddenly finding that they can reach out to a global customer base. However, with a broader global customer base also comes worldwide competition. More competition impacts the way developers need to think about building applications. For example:

- *Complexity has gone way up.* Customers expect that all parts of an organization know who they are. But "siloed" applications that talk to a single database and don't integrate with other applications are no longer the norm. Today's applications need to communicate with multiple services and databases, residing not only inside a company's data center but also within external internet service providers.
- *Customers want faster delivery.* Customers no longer want to wait for the next annual release of a software package. Instead, they expect the features in a software product to be unbundled so that new functionality can be released quickly in a matter of weeks (or even days).
- *Customers also demand reliable performance and scalability.* Global applications make it extremely difficult to predict how many transactions are going to be handled by an application and when that transaction volume is going to hit. Applications need to scale up quickly across multiple servers, then scale back down seamlessly when the volume has passed.
- *Customers expect their applications to be available.* Because customers are one click away from a competitor, a company's applications must be highly resilient. Failures or problems in one part of the application shouldn't bring down the entire application.

To meet these expectations, we, as application developers, have to embrace the enigma that to build highly scalable and highly redundant applications, we need to break our applications into small services that can be built and deployed independently of one another. If we "unbundle" our applications into smaller services and move these away from a single monolithic artifact, we can build systems that are

- *Flexible*—Decoupled services can be composed and rearranged to quickly deliver new functionality. The smaller the unit of code that one is working with, the less complicated it is to change and the less time it takes to test and deploy the code.
- *Resilient*—Decoupled services mean an application is no longer a single "ball of mud," where a degradation in one part of the application causes the entire application to fail. Failures can be localized to a small part of the application and contained before the entire application shuts down. This also enables the application to degrade gracefully in case of an unrecoverable error.

- *Scalable*—Decoupled services can easily be distributed horizontally across multiple servers, making it possible to scale the features/services appropriately. With a monolithic application, where all the logic for the application is intertwined, the *entire* application needs to scale back, even if only a small part of the application is the bottleneck. With small services, scaling is localized and much more cost effective.

To this end, we begin our discussion of microservices. Keep the following in mind as we start our journey:

Small, Simple, and Decoupled Services = Scalable, Resilient, and Flexible Applications

It's important to understand that systems and organizations can benefit from a microservices approach. To obtain benefits in the organization, we can apply *Conway's law* in reverse. This law indicates several points that can improve the communication and structure of a company.

Conway's law (which first appeared in April, 1968, written by Melvin R. Conway in the article, "How Do Committees Invent") states that "Organizations which design systems . . . are constrained to produce designs which are copies of the communication structures of these organizations." Basically, what that indicates is that the way teams communicate within the team and with other teams is directly reflected in the code they produce.

If we apply Conway's law in reverse (also known as *inverse Conway maneuver*) and design the company structure based on a microservice architecture, the communication, stability, and organizational structure of our applications improve by creating loosely coupled and autonomous teams to implement the microservices.

1.2 Microservices with Spring

Spring has become the most popular development framework for building Java-based applications. At its core, Spring is based on the concept of dependency injection. A *dependency injection framework* allows you to more efficiently manage large Java projects by externalizing the relationship between objects within your application through convention (and annotations) rather than hardcoding those objects to "know" about each other. Spring sits as an intermediary between the different Java classes of your application and manages their dependencies. Spring essentially lets you assemble your code like a set of Lego bricks that snap together.

What's impressive about the Spring framework, and a testament to its development community, is its ability to stay relevant and to reinvent itself. The Spring developers quickly saw that many development teams were moving away from monolithic applications where the application's presentation, business, and data access logic were packaged together and deployed as a single artifact. Instead, they noticed that development teams were moving to highly distributed models where small services can be quickly deployed to the cloud. In response to this shift, the Spring developers launched two projects: Spring Boot and Spring Cloud.

Spring Boot is a re-envisioning of the Spring framework. While it embraces core features of Spring, Spring Boot strips away many of the "enterprise" features found in Spring and instead delivers a framework geared toward Java-based, REST-oriented (Representational State Transfer) microservices. With a few simple annotations, a Java developer can quickly build a REST service that can be packaged and deployed without the need for an external application container.

NOTE While we cover REST in more detail in chapter 3, the core concept behind REST is that your services should embrace the use of HTTP verbs (GET, POST, PUT, and DELETE) to represent the core actions of the service and should use a lightweight, web-oriented data serialization protocol, such as JSON, for requesting and receiving data from the service.

The key features of Spring Boot include the following:

- An embedded web server to avoid complexity in deploying the application: Tomcat (default), Jetty, or Undertow.
  This is one essential component of Spring Boot; the chosen web server is part of the deployable JAR. For Spring Boot applications, the only requisite to deploy the app is to have Java installed on the server.
- A suggested configuration to start quickly with a project (starters).
- An automatic configuration for Spring functionally—whenever it's possible.
- A wide range of features ready for production (such as metrics, security, status verification, externalized configuration, and more).

Using Spring Boot offers the following benefits for our microservices:

- Reduces development time and increases efficiency and productivity
- Offers an embedded HTTP server to run web applications
- Allows you to avoid writing a lot of boilerplate code

- Facilitates integration with the Spring Ecosystem (includes Spring Data, Spring Security, Spring Cloud, and more)
- Provides a set of various development plugins

Because microservices have become one of the more common architectural patterns for building cloud-based applications, the Spring development community gives us Spring Cloud. The Spring Cloud framework makes it simple to operationalize and deploy microservices to a private or public cloud. Spring Cloud wraps several popular cloud-management microservice frameworks in a common framework. It makes the use and deployment of these technologies as easy as is annotating your code. We cover the different components within Spring Cloud in the next chapter.

1.3 What are we building?

This book offers a step-by-step guide on creating a complete microservices architecture using Spring Boot, Spring Cloud, and other useful and modern technologies. Figure 1.4 shows a high-level overview of some of the services and technology integrations that we will use throughout the book.

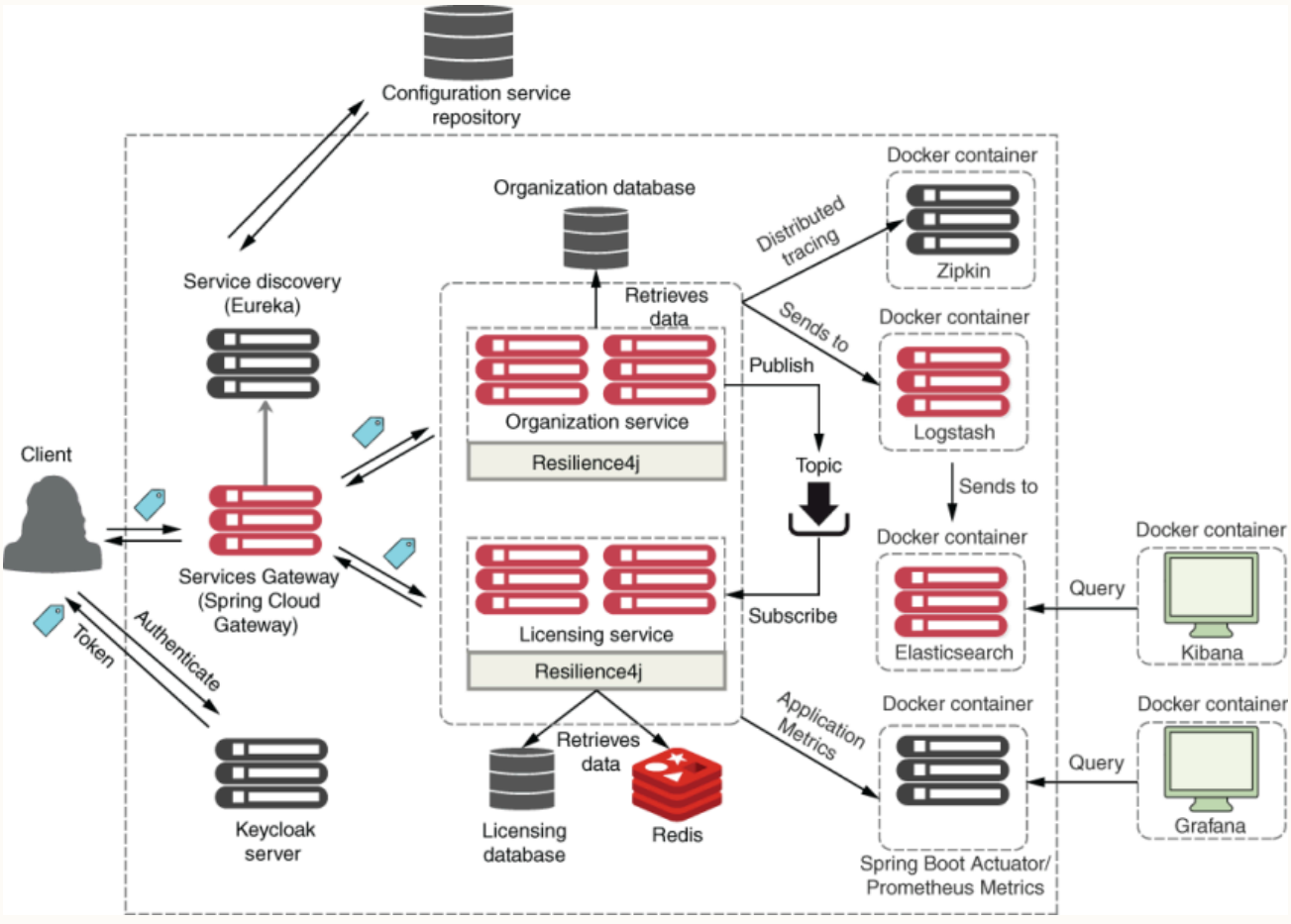


Figure 1.4 High-level overview of the services and technologies that we'll use in this book

Figure 1.4 describes a client request to update and retrieve the organization's information in the microservice architecture that we will create. To start the request, the client first needs to authenticate with Keycloak to get an access token. Once the token is obtained, the client makes a request to the Spring Cloud API Gateway. The API Gateway service is the entry point to our entire architecture; this service communicates with the Eureka service discovery to retrieve the locations of the organization and licensing services and then calls the specific microservice.

Once the request arrives at the organization service, it validates the access token against Keycloak to see if the user has permission to continue the process. Once validated, the organization service updates and retrieves its information from the organization database and sends it back to the client as an HTTP response. As an alternative path, once the organization information is updated, the organization service adds a message to the Kafka topic so the licensing service is aware of the change.

When the message arrives at the licensing service, Redis stores the specific information in Redis's in-memory database. Throughout this process, the architecture uses distributed tracing from Zipkin, Elasticsearch, and Logstash to manage and display the logs and employs Spring Boot Actuator, Prometheus, and Grafana to expose and display the application metrics.

As we move forward, we will see topics such as Spring Boot, Spring Cloud, Elasticsearch, Logstash, Kibana, Prometheus, Grafana, and Kafka, among others. All these technologies may sound complicated, but we will see how to create and integrate the different components that make up the diagram in figure 1.4 as we progress through the book.

1.4 What is this book about?

The scope of this book is broad. It covers everything from basic definitions to more complex implementations to create a microservices architecture.

4.1 What you'll learn in this book

This book is about building microservice-based applications using a variety of Spring projects, such as Spring Boot and Spring Cloud, which can be deployed locally in a private cloud run by your company or in a public cloud such as Amazon, Google, or Azure. This book covers the following topics:

- What a microservice is, best practices, and design considerations that go into building a microservice-based application
- When you shouldn't build a microservice-based application

- How to build microservices using the Spring Boot framework
- The core operational patterns to support microservice applications, particularly a cloud-base application
- What Docker is and how to integrate it with a microservice-based application
- How you can use Spring Cloud to implement the operational patterns described later in this chapter
- How to create application metrics and visualize these in a monitoring tool
- How to achieve distributed tracing with Zipkin and Sleuth
- How to manage application logs with the ELK Stack
- How to take what you've learned and build a deployment pipeline that can be used to deploy your services locally, to a private internally managed cloud, or to a public cloud provider

By the time you're done reading this book, you should have the knowledge needed to build and deploy a Spring Boot microservice. You'll also understand the key design decisions needed to operationalize your microservices. You'll realize how service configuration management, service discovery, messaging, logging and tracing, and security all fit together to deliver a robust microservice environment. Finally, you'll see how your microservices can be deployed using different technologies.

4.2 Why is this book relevant to you?

I suspect that if you have reached this point, it is because you

- Are a Java developer or have a strong grasp of Java
- Have a background in Spring
- Are interested in learning how to build microservice-based applications
- Are interested in how to use microservices to build cloud-based applications
- Want to know if Java and Spring are relevant technologies for building microservice-based applications
- Want to know what the cutting-edge technologies are to achieve a microservice architecture
- Are interested in seeing what goes into deploying a microservice-based application to the cloud

This book offers a detailed guide on how to implement a microservices architecture in Java. It provides descriptive and visual information and a lot of hands-on code examples to give you a programmatic guide on how to implement this architecture using the latest versions of different Spring projects like Spring Boot and Spring Cloud.

Additionally, this book provides an introduction to the microservice patterns, best practices, and infrastructure technologies that go hand in hand with this type of architecture, simulating a real-world application development environment. Let's shift gears for a moment and walk through building a simple microservice using Spring Boot.

1.5 Cloud and microservice-based applications

In this section, we'll see how to create a microservice using Spring Boot and learn why the cloud is relevant to microservice-based applications.

5.1 Building a microservice with Spring Boot

This section will not provide a detailed walk-through of much of the code you'll use to create microservices, but is just a brief introduction on how to create a service to show you how easy it is to use Spring Boot. For this, we're going to create a simple REST service for "Hello World" with one main endpoint that uses the GET HTTP verb. This service endpoint will receive request parameters and URL parameters (also known as *path variables*). Figure 1.5 shows what the REST service will do and the general flow of how a Spring Boot microservice processes a user's request.

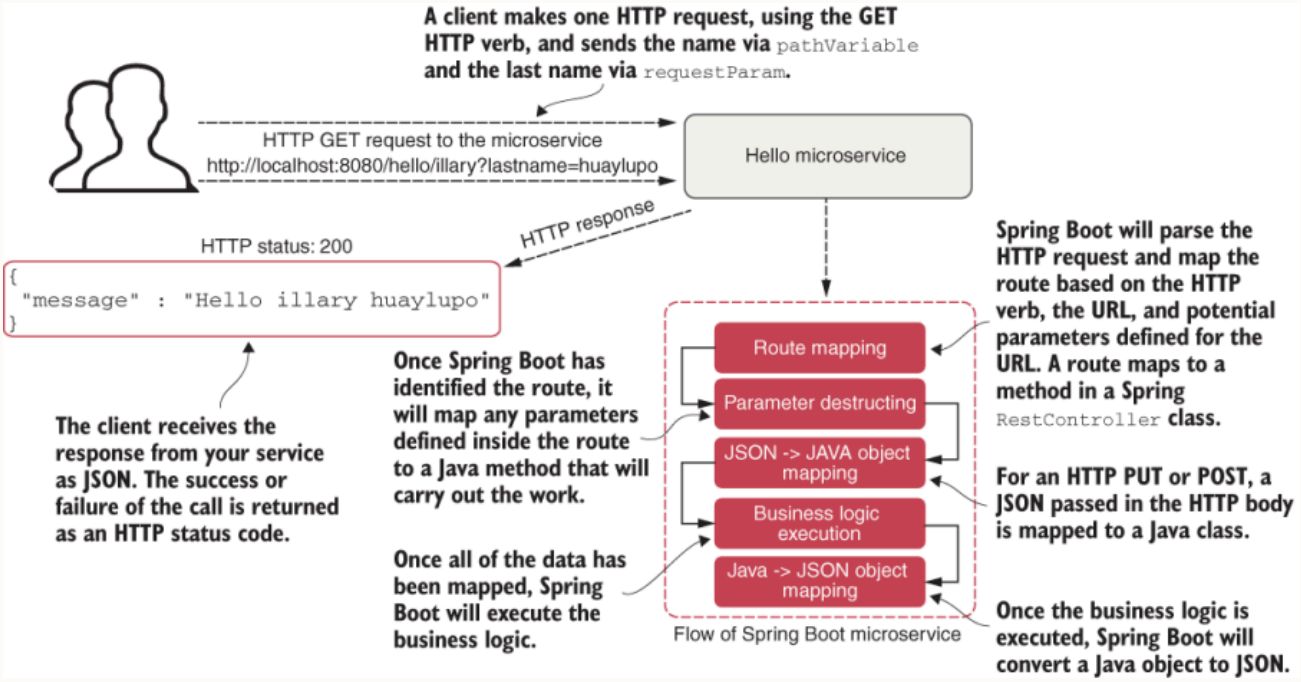


Figure 1.5 Spring Boot abstracts away the common REST microservice tasks (routing to business logic, parsing HTTP parameters from the URL, mapping JSON to and from Java objects) and lets the developer focus on the business logic for the service. This figure shows three different ways to pass parameters to our controller.

This example is by no means exhaustive or even illustrative of how you should build a production-level microservice, but it should cause you to pause because of how little

code it takes to write it. We won't go through how to set up the project build files or the details of the code until chapter 2. If you'd like to see the Maven pom.xml file and the actual code, you can find it in the chapter 1 section of the downloadable code.

NOTE You can retrieve all the source code for chapter 1 from the GitHub repository for the book at **https://github.com/ihuaylupo/manning-smia/tree/master/chapter1**.

For this example, we have a single Java class called `Application`, which you'll find in the class file, com/huaylupo/spmia/ch01/SimpleApplication/Application.java. We will use this class to expose a REST endpoint called `/hello`. The following listing shows the code for the `Application` class.

Listing 1.1 Hello World with Spring Boot: a (very) simple Spring microservice

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication                          (1)
@RestController                                 (2)
@RequestMapping(value="hello")                  (3)
public class Application {

  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }

  @GetMapping(value="/{firstName}")             (4)
  public String helloGET(
    @PathVariable("firstName") String firstName,    (5)
    @RequestParam("lastName") String lastName) {    (5)
    return String.format(
        "{\"message\":\"Hello %s %s\"}",        (6)
          firstName, lastName);
  }
}

class HelloRequest{                             (7)
```

```
    private String firstName;
    private String lastName;

    public String getFirstName() {
      return firstName;
    }
    public void setFirstName(String firstName) {
      this.firstName = firstName;
    }
    public String getLastName() {
      return lastName;
    }
    public void setLastName(String lastName) {
      this.lastName = lastName;
    }

  }
```

❶ Tells Spring Boot that this class is the entry point for the Spring Boot service

❷ Tells Spring Boot to expose the code in this class as a Spring RestController

❸ Prefaces all URLs exposed in this application with a /hello prefix

❹ Exposes an endpoint as a GET-based REST that takes two parameters in its firstName (via @PathVariable) and lastName (via @RequestParam)

❺ Maps the firstName and lastName parameters to the two variables passed into the hello function

❻ Returns a simple JSON string that we manually build (in chapter 2, we won't create any JSON)

❼ Contains the fields of the JSON structure sent by the user

In listing 1.1, you're basically exposing one endpoint with a GET HTTP verb that takes two parameters (`firstName` and `lastName`) on the URL: one from the path variable (`@PathVariable`) and another one as the request parameter (`@RequestParam`). The endpoint returns a simple JSON string that has a payload containing the message `"Hello firstName lastName"` . To call the GET endpoint `/hello/illary?lastName=huaylupo` on your service, the return call would be

```
{"message":"Hello illary huaylupo"}
```

Let's start the Spring Boot application. In order to do this, let's execute the following command on the command line. This Maven command uses a Spring Boot plugin defined in the pom.xml file to start the application using an embedded Tomcat server. Once you execute the `mvn spring-boot:run` command and everything starts correctly, you should see what's shown in figure 1.6 in your command-line window.

```
mvn spring-boot:run
```

NOTE If you are running the command from the command line, make sure you are in the root directory. The root directory is the one that contains the pom.xml file. Otherwise, you will run into this error: No plugin found for prefix 'spring-boot' in the current project and in the plugin groups.

Java vs. Groovy and Maven vs. Gradle

The Spring Boot framework supports both Java and the Groovy programming languages. Spring Boot also supports both Maven and Gradle build tools. Gradle introduces a Groovy-based DSL (domain specific language) to declare the project configuration instead of an XML file like Maven. Although Gradle is powerful, flexible, and top-rated, Maven is still used by the Java developer community. This book, therefore, only contains examples in Maven to keep it manageable and the material focused, and it is intended to reach the largest audience possible.

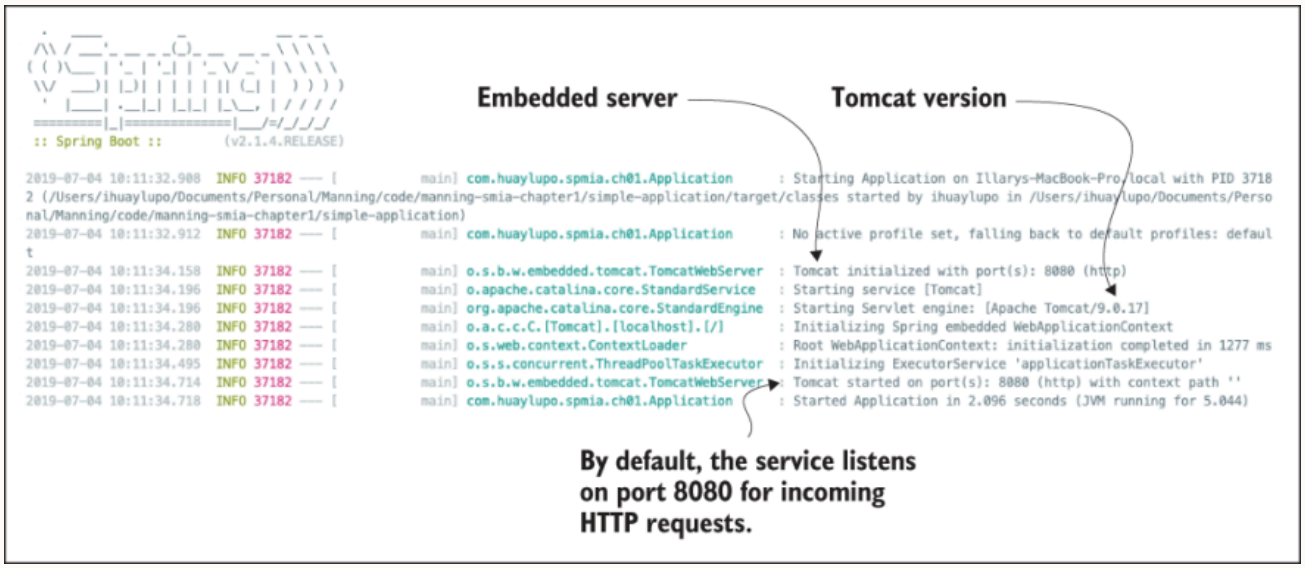


Figure 1.6 The Spring Boot service communicates the service port via the console.

To execute the services, you need to use a browser-based REST tool. You'll find many tools, both graphical and command-line, for invoking REST-based services. For this book, we will use Postman (https://www.getpostman.com/). Figures 1.7 and 1.8 show two different Postman calls to the endpoints with the results returned from the services.

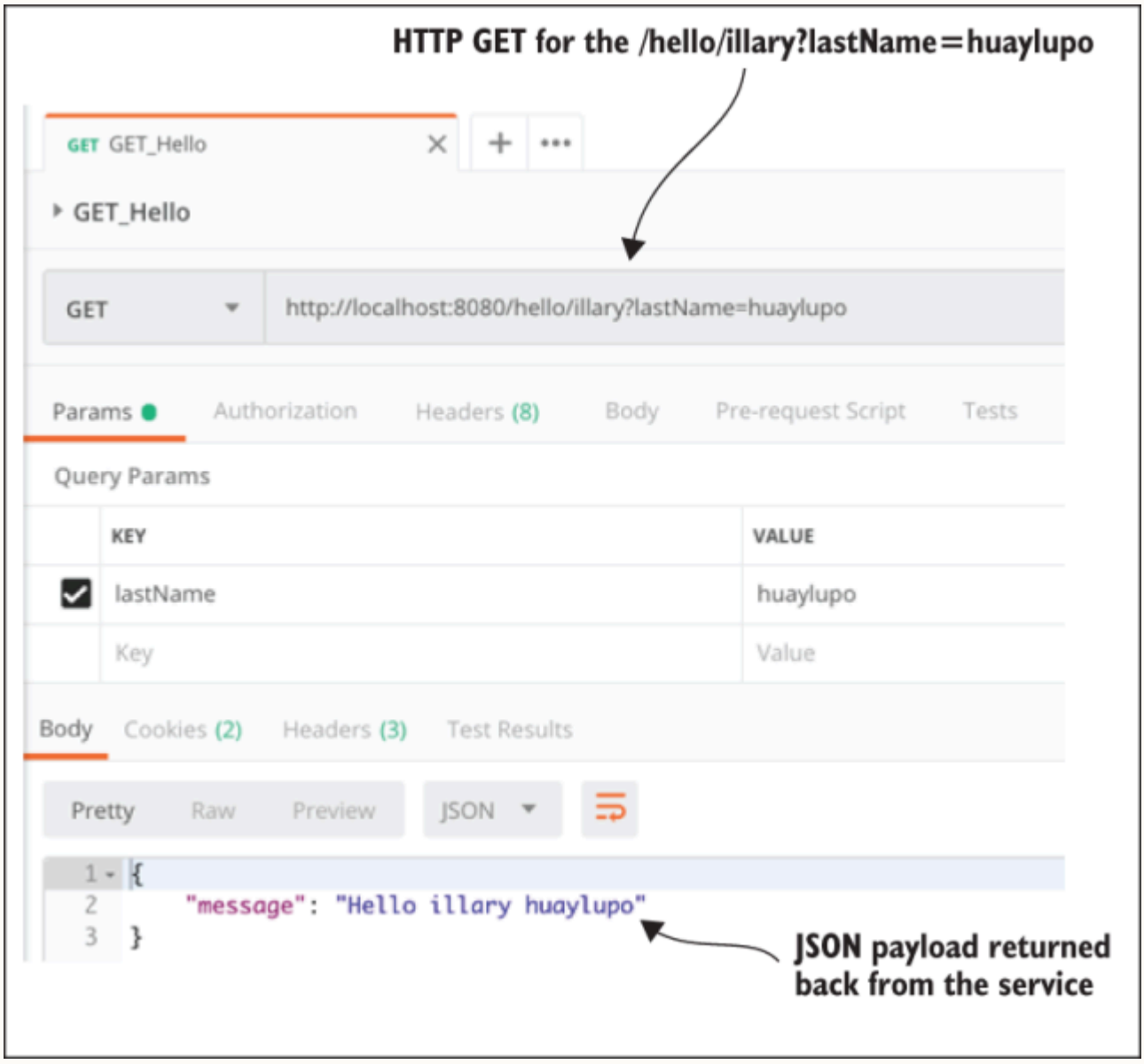


Figure 1.7 The response from the GET `/hello` endpoint shows the data you've requested represented as a JSON payload.

Figure 1.8 shows a brief example of how to make a call using the POST HTTP verb. It is essential that we mention this is only for demonstration purposes. In the following chapters, you'll see that the POST method is preferred when it involves creating new records in our service.

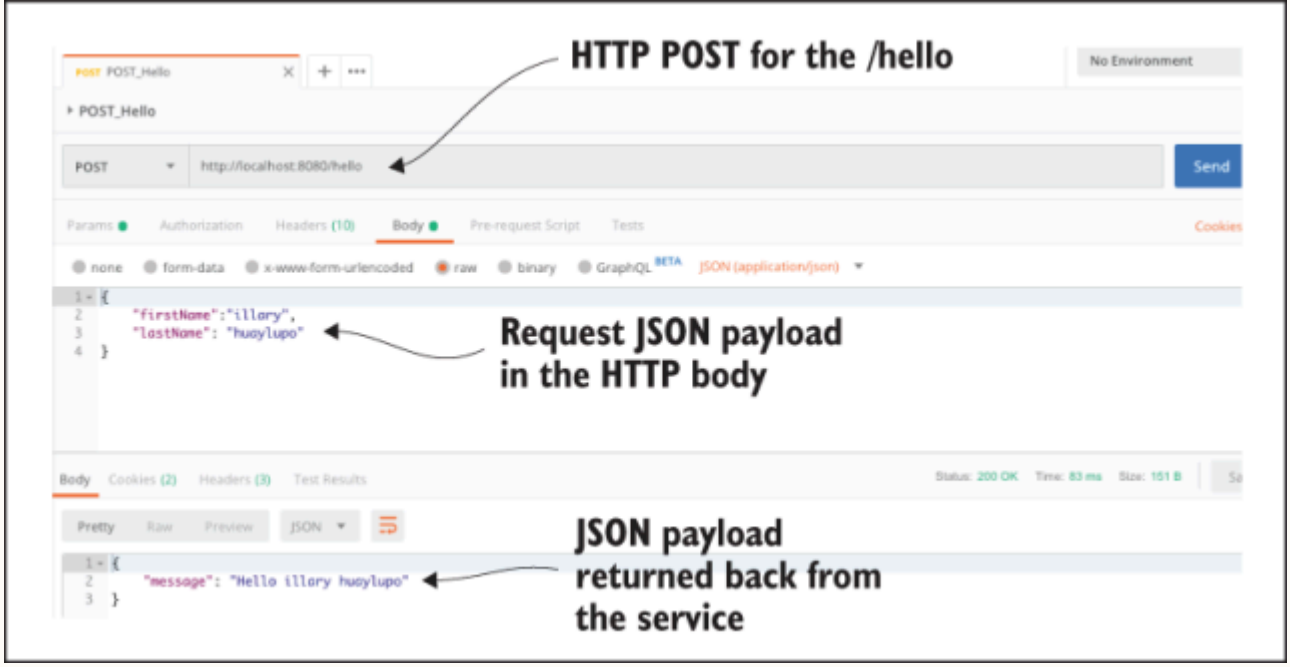


Figure 1.8 The response from the POST `/hello` endpoint shows the request and the response data represented as a JSON payload.

This simple example code doesn't demonstrate the full power of Spring Boot, nor the best practices to create a service. But what it shows is that you can write a full HTTP JSON REST-based service with route-mapping of the URL and parameters in Java with a few lines of code. Although Java is a powerful language, it has acquired a reputation for being wordy compared with other languages. With Spring, however, we can accomplish a lot with just a few lines of code. Next, let's walk through why and when a microservice approach is justified for building applications.

5.2 What exactly is cloud computing?

Cloud computing is the delivery of computing and virtualized IT services—databases, networking, software, servers, analytics, and more—through the internet to provide a flexible, secure, and easy-to-use environment. Cloud computing offers significant advantages in the internal management of a company, such as low initial investment, ease of use and maintenance, and scalability, among others.

The cloud computing models let the user choose the level of control over the information and services that these provide. These models are known by their acronyms, and are generically referred to as XaaS —an acronym that means anything as a service. The following lists the most common cloud computing models. Figure 1.9 shows the differences between these models.

- *Infrastructure as a Service (IaaS)*—The vendor provides the infrastructure that lets you access computing resources such as servers, storage, and networks. In this model, the user is responsible for everything related to the maintenance of the infrastructure and the scalability of the application.
  IaaS platforms include AWS (EC2), Azure Virtual Machines, Google Compute Engine, and Kubernetes.
- *Container as a Service (CaaS)*—An intermediate model between the IaaS and the PaaS, it refers to a form of container-based virtualization. Unlike an IaaS model, where a developer manages the virtual machine to which the service is deployed, with CaaS, you deploy your microservices in a lightweight, portable virtual container (such as Docker) to a cloud provider. The cloud provider runs the virtual server the container is running on, as well as the provider's comprehensive tools for building, deploying, monitoring, and scaling containers.
  CaaS platforms include Google Container Engine (GKE) and Amazon's Elastic Container Service (ECS). In chapter 11, we'll see how to deploy the microservices you've built to Amazon ECS.
- *Platform as a Service (PaaS)*—This model provides a platform and an environment that allow users to focus on the development, execution, and maintenance of the application. The applications can be created with tools that are provided by the vendor (for example, operating system, database management systems, technical support, storage, hosting, network, and more). Users do not need to invest in a physical infrastructure, nor spend time managing it, allowing them to concentrate exclusively on the development of applications.
  PaaS platforms include Google App Engine, Cloud Foundry, Heroku, and AWS Elastic Beanstalk.
- *Function as a Service (FaaS)*—Also known as serverless architecture, despite the name, this architecture doesn't mean running specific code without a server. What it means is a way of executing functionalities in the cloud in which the vendor provides all the required servers. Serverless architecture allows us to focus only on the development of services without having to worry about scaling, provisioning, and server administration. Instead, we can solely concentrate on uploading our functions without handling any administration infrastructure.
  FaaS platforms include AWS (Lambda), Google Cloud Function, and Azure functions.
- *Software as a Service (SaaS)*—Also known as software on demand, this model allows users to use a specific application without having to deploy or to maintain it. In most cases, the access is through a web browser. Everything is managed by the service provider: application, data, operating system, virtualization, servers, storage, and network. The user just hires the service and uses the software.
  SaaS platforms include Salesforce, SAP, and Google Business.

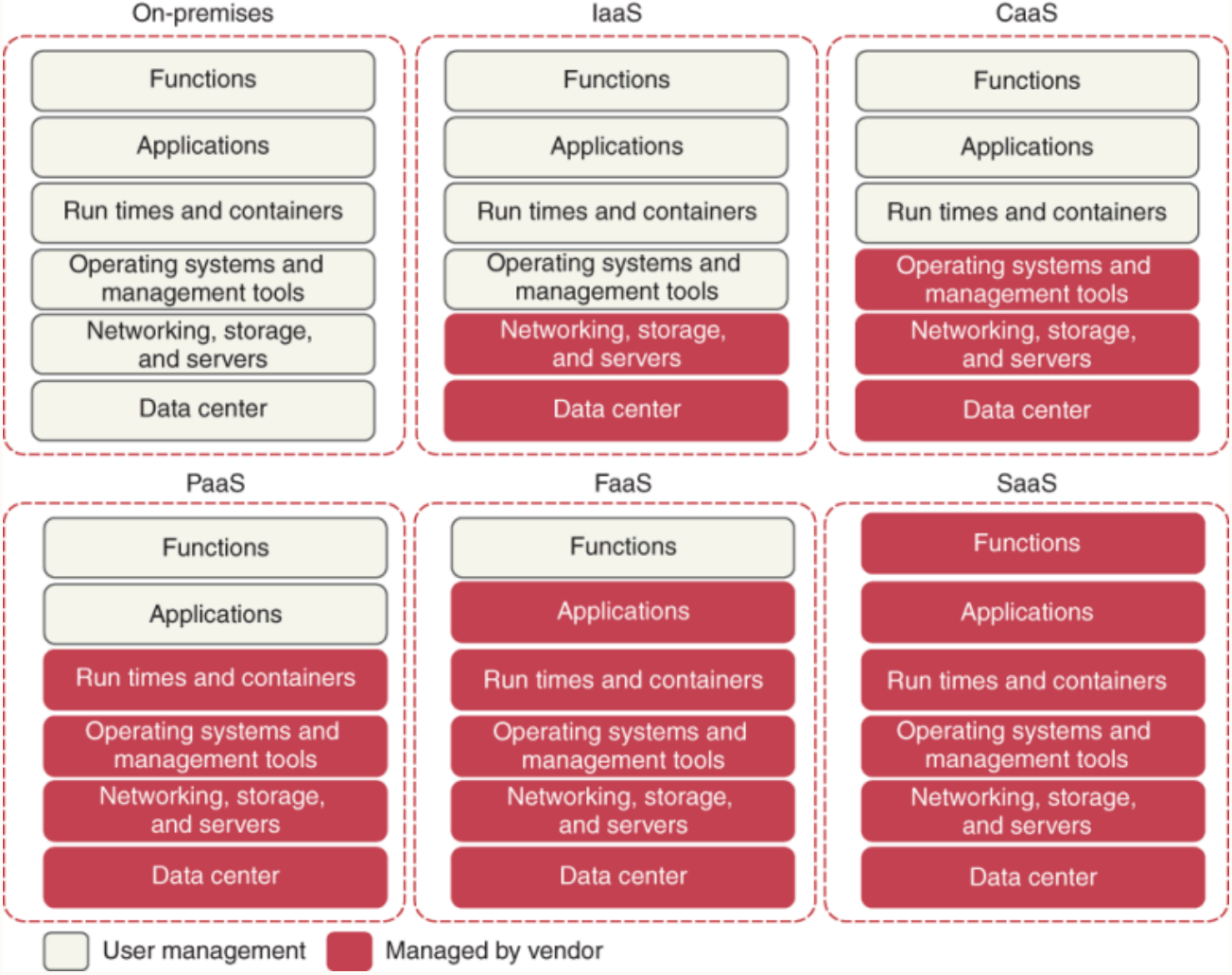


Figure 1.9 The different cloud computing models come down to who's responsible for what: user management or cloud vendor.

NOTE If you're not careful, FaaS-based platforms can lock your code into a cloud vendor platform because your code is deployed to a vendor-specific runtime engine. With a FaaS-based model, you might be writing your service using a general programming language (Java, Python, JavaScript, and so on), but you're still tying yourself to the underlying vendor's APIs and runtime engine that your function will be deployed to.

5.3 Why the cloud and microservices?

One of the core concepts of a microservice architecture is that each service is packaged and deployed as its own discrete and independent artifact. Service instances should be brought up quickly, and each should be indistinguishable from another. When writing a microservice, sooner or later you're going to have to decide whether your service is going to be deployed to one of the following:

- *Physical server*—While you can build and deploy your microservices to a physical machine(s), few organizations do this because physical servers are constrained. You can't quickly ramp up the capacity of a physical server, and it can become extremely costly to scale your microservice horizontally across multiple physical servers.

- *Virtual machine images*—One of the key benefits of microservices is their ability to quickly start up and shut down instances in response to scalability and service failure events. Virtual machines (VMs) are the heart and soul of the major cloud providers.
- *Virtual container*—Virtual containers are a natural extension of deploying your microservices on a VM image. Rather than deploying a service to a full VM, many developers deploy their services as Docker containers (or equivalent container technology) to the cloud. Virtual containers run inside a VM, and using a virtual container, you can segregate a single VM into a series of self-contained processes that share the same image. A microservice can be packaged, and multiple instances of the service can then be quickly deployed and started in either an IaaS private or public cloud.

The advantage of cloud-based microservice centers around the concept of *elasticity*. Cloud service providers allow you to quickly spin up new VMs and containers in a matter of minutes. If your capacity needs for your services drop, you can spin down containers to avoid additional costs. Using a cloud provider to deploy your microservices gives you significantly more horizontal scalability (adding more servers and service instances) for your applications.

Server elasticity also means that your applications can be more resilient. If one of your microservices is having problems and is failing over, spinning up new service instances can keep your application alive long enough for your development team to gracefully resolve the issue.

For this book, all the microservices and corresponding service infrastructure will be deployed to a CaaS-based cloud provider using Docker containers. This is a common deployment topology for microservices. The most common characteristics of CaaS cloud providers are as follows:

- *Simplified infrastructure management*—CaaS cloud providers give you the ability to have more control over your services. New services can be started and stopped with simple API calls.
- *Massive horizontal scalability*—CaaS cloud providers allow you to quickly and succinctly start one or more instances of a service. This capability means you can quickly scale services and route around misbehaving or failing servers.
- *High redundancy through geographic distribution*—By necessity, CaaS providers have multiple data centers. By deploying your microservices using a CaaS cloud provider, you can gain a higher level of redundancy beyond using clusters in a data center.

Why not PaaS-based microservices?

Earlier in the chapter I discussed five types of cloud platforms—Infrastructure as a Service (IaaS), Container as a Service (CaaS), Platform as a Service (PaaS), Function as a Service (FaaS), and Software as a Service (SaaS). This book focuses specifically on building microservices using a CaaS approach. While certain cloud providers will let you abstract away the deployment infrastructure of your microservice, this book will teach you how to remain vendor independent and deploy all parts of the application (including the servers).

For instance, Cloud Foundry, AWS Elastic Beanstalk, Google App Engine, and Heroku give you the ability to deploy your services without having to know about the underlying application container. These provide a web interface and command-line interface (CLI) to allow you to deploy your application as a WAR or JAR file. Setting up and tuning the application server and the corresponding Java container are abstracted away from you. While this is convenient, each cloud provider's platform has different idiosyncrasies related to its individual PaaS solution.

The services built in this book are packaged as Docker containers; the main reason is that Docker is deployable to all major cloud providers. In later chapters, we'll see what Docker is and learn how to integrate Docker to run all the services and infrastructure used in this book.

1.6 Microservices are more than writing the code

While the concepts around building individual microservices are easy to understand, running and supporting a robust microservice application (especially when running in the cloud) involves more than just writing the code for the service. Figure 1.10 shows some guidelines to consider while writing or building a microservice.

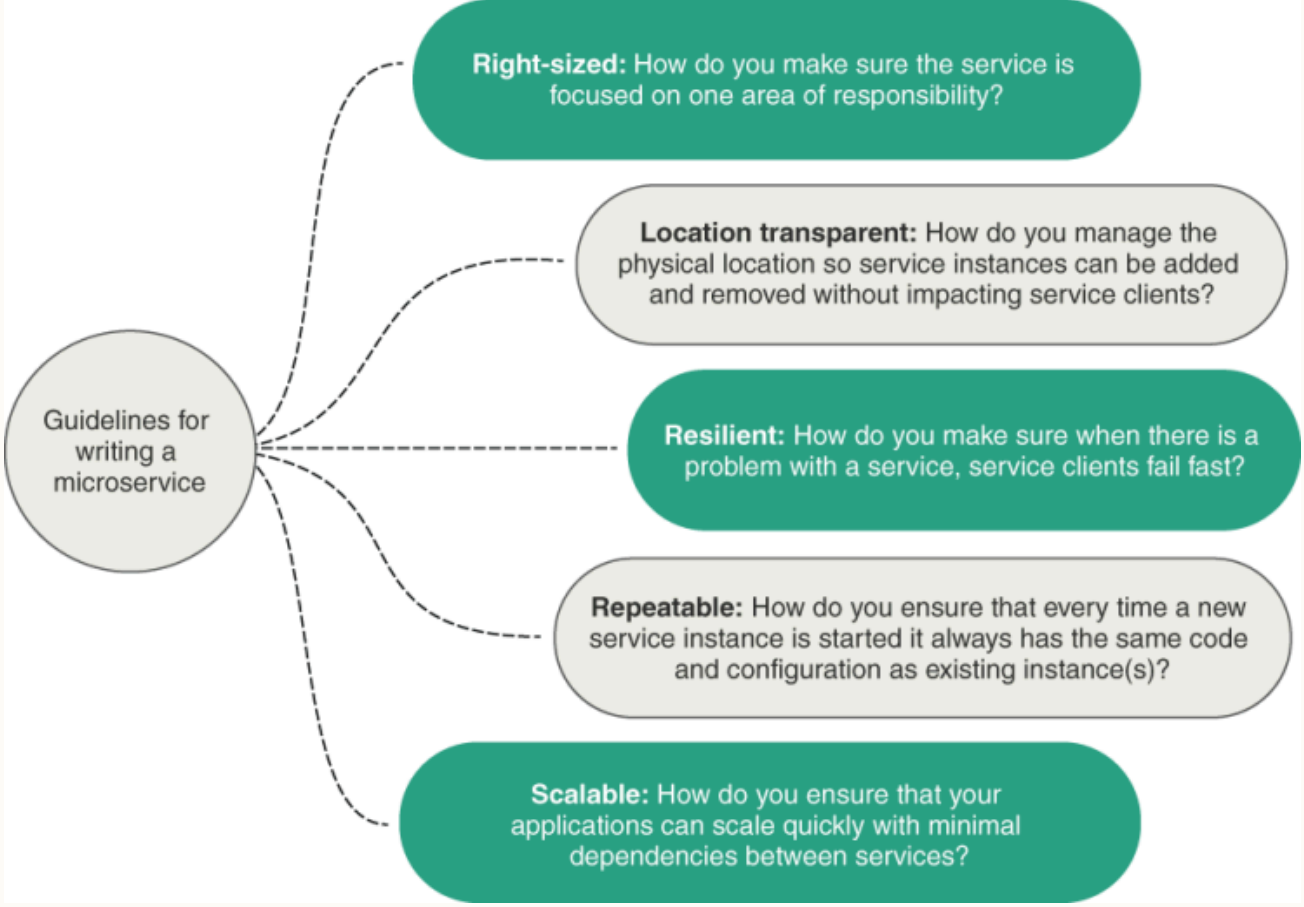


Figure 1.10 Microservices are more than the business logic. You need to think about the environment where you'll run the services and how the services will scale and be resilient.

Writing a robust service includes considering several topics. Let's walk through the items show in figure 1.10 in more detail:

- *Right-sized*—How you ensure that your microservices are properly sized so that you don't have a microservice take on too much responsibility. Remember, properly sized, a service allows you to make changes to an application quickly and reduces the overall risk of an outage to the entire application.
- *Location transparent*—How you manage the physical details of service invocation. When in a microservice application, multiple service instances can quickly start and shut down.
- *Resilient*—How you protect your microservice consumers and the overall integrity of your application by routing around failing services and ensuring that you take a "fail-fast" approach.
- *Repeatable*—How you ensure that every new instance of your service brought up is guaranteed to have the same configuration and codebase as all the other service instances in production.
- *Scalable*—How you establish a communication that minimizes the direct dependencies between your services and ensures that you can gracefully scale your microservices.

This book takes a patterns-based approach as we look at these items in more detail. With a patterns-based approach, we'll look at common designs that can be used across different technology implementations. While we've chosen to use Spring Boot and Spring Cloud to implement the patterns we're going to use in this book, nothing will keep you from taking the concepts presented here and using these with other technology platforms. Specifically, we'll cover the following microservice patterns:

- Core development pattern
- Routing patterns
- Client resiliency patterns
- Security patterns
- Logging and tracing patterns
- Application metrics patterns
- Build and deployment pattern

It's important to understand that there isn't a formal definition of how to create a microservice. In the next section, you'll see a list of common aspects you need to take into consideration while building a microservice.

1.7 Core microservice development pattern

The core microservice development pattern addresses the basics of building a microservice. Figure 1.11 highlights the topics we'll cover around basic service design.

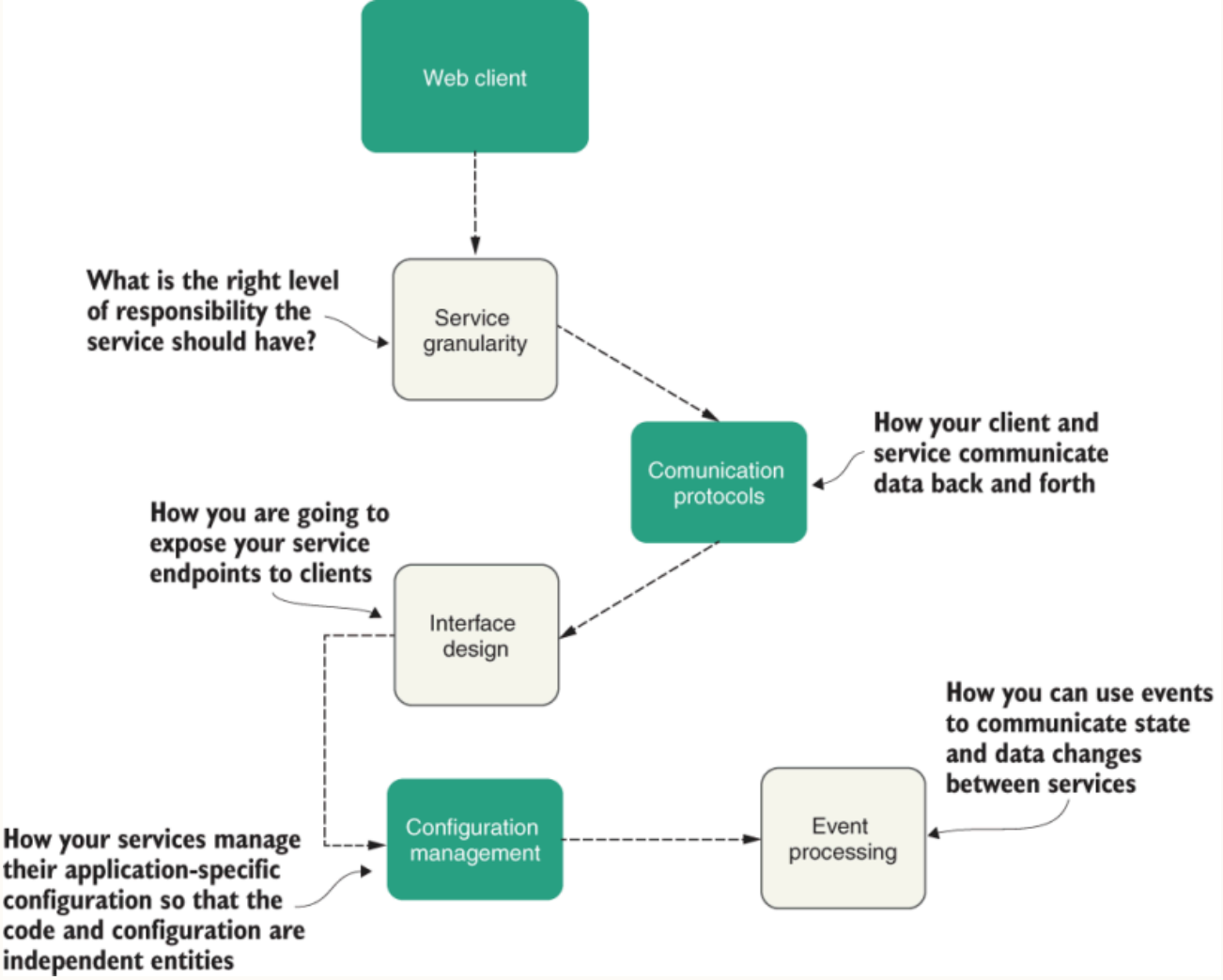


Figure 1.11 When designing your microservice, you need to think about how the service will be consumed and communicated with.

The following patterns (shown in figure 1.11) show the basics of building a microservice:

- *Service granularity*—How do you approach decomposing a business domain down into microservices so that each microservice has the right level of responsibility? Making a service too coarse-grained with responsibilities that overlap into different business-problems domains makes the service difficult to maintain and change over time. Making the service too fine-grained increases the overall complexity of the application and turns the service into a "dumb" data abstraction layer with no logic except for that needed to access the data store. Service granularity is covered in chapter 3.

- *Communication protocols*—How will developers communicate with your service? The first step is to define whether you want a synchronous or asynchronous protocol. For synchronous, the most common communication is HTTP-based REST using XML (Extensible Markup Language), JSON (JavaScript Object Notation), or a binary protocol such as Thrift to send data back and forth to your microservices. For asynchronous, the most popular protocol is AMQP (Advanced Message Queuing Protocol) using a one-to-one (queue) or a one-to-many (topic) with message brokers such as RabbitMQ, Apache Kafka, and Amazon Simple Queue Service (SQS). In later chapters, we'll learn about the communication protocols.
- *Interface design*—What's the best way to design the actual service interfaces that developers are going to use to call your service? How do you structure your services? What are the best practices? Best practices and interface design are covered in the next chapters.
- *Configuration management of service*—How do you manage the configuration of your microservice so that it moves between different environments in the cloud? This can be managed with externalized configuration and profiles as seen in chapter 5.
- *Event processing between services*—How do you decouple your microservice using events so that you minimize hardcoded dependencies between your services and increase the resiliency of your application? We'll use an event-driven architecture with Spring Cloud Stream as covered in chapter 10.

1.8 Microservice routing patterns

The microservice routing patterns deal with how a client application that wants to consume a microservice discovers the location of the service and is routed over to it. In a cloud-based application, it is possible to have hundreds of microservice instances running. To enforce security and content policies, it is required to abstract the physical IP address of those services and have a single point of entry for the service calls. How? The following patterns are going to answer that question:

- *Service discovery*—With service discovery and its key feature, service registry, you can make your microservice discoverable so client applications can find them without having the location of the service hardcoded into their application. How? We explain this in chapter 6. Remember the service discovery is an internal service, not a client-facing service.
  Note that in this book, we use Netflix Eureka Service Discovery, but there are other service registries such as etcd, Consul, and Apache Zookeeper. Also, some systems do not have an explicit service registry. Instead these use an interservice communication infrastructure known as a *service mesh*.

- *Service routing*—With an API Gateway, you can provide a single entry point for all of your services so that security policies and routing rules are applied uniformly to multiple services and service instances in your microservices applications. How? With the Spring Cloud API Gateway, as we explain in chapter 8.

Figure 1.12 shows how service discovery and service routing appear to have a hardcoded sequence of events between them (first comes service routing and then service discovery). However, the two patterns aren't dependent on one another. For instance, we can implement service discovery without service routing, and we can implement service routing without service discovery (even though its implementation is more difficult).

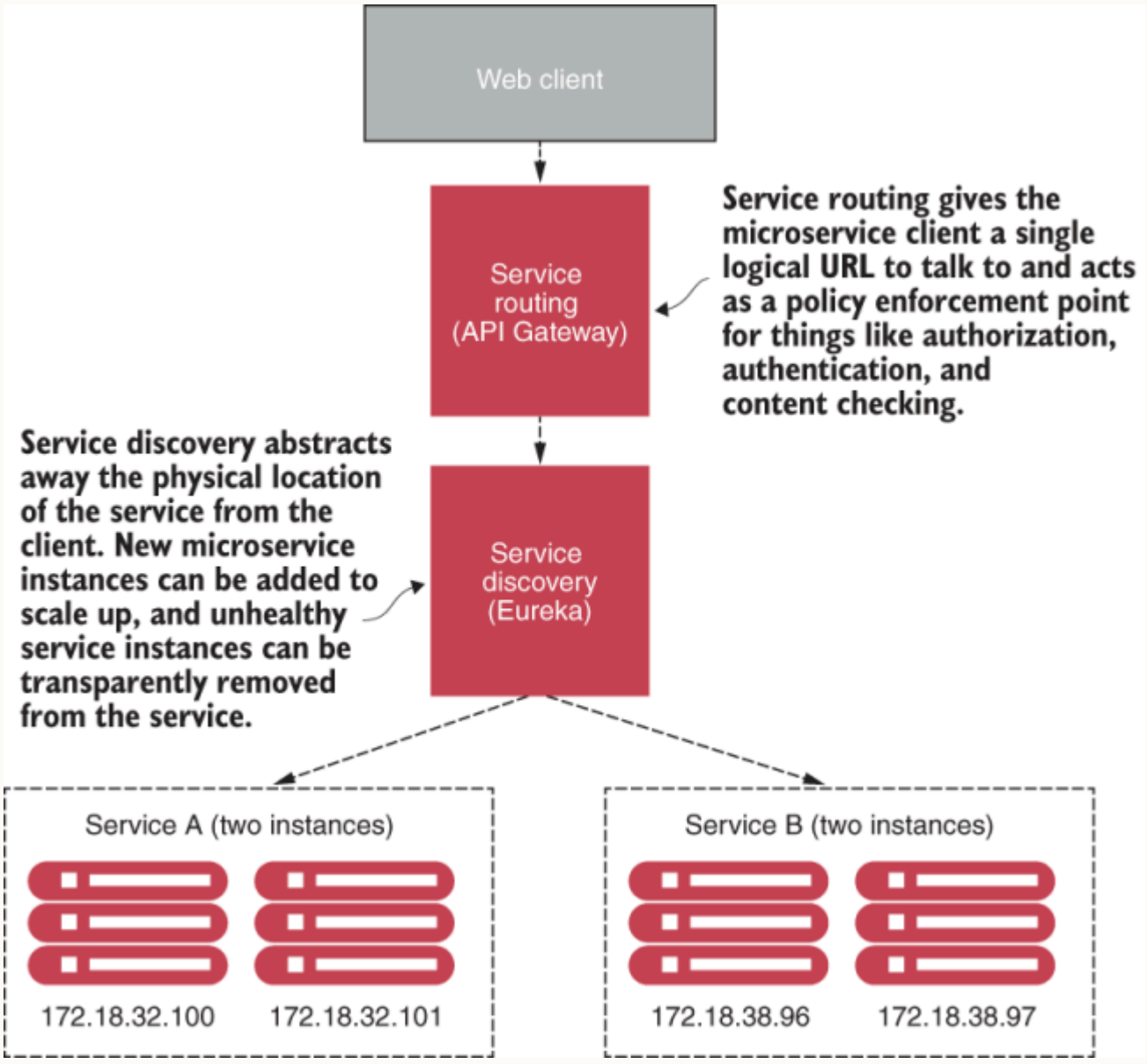


Figure 1.12 Service discovery and service routing are key parts of any large-scale microservice application.

1.9 Microservice client resiliency

Because microservice architectures are highly distributed, you have to be extremely sensitive in how you prevent a problem in a single service (or service instance) from cascad-

ing up and out to the consumers of the service. To this end, we'll cover four client resiliency patterns:

- *Client-side load balancing*—How you cache the location of your service instances on the service so that calls to multiple instances of a microservice are load balanced to all the health instances of that microservice.
- *Circuit breaker pattern*—How you prevent a client from continuing to call a service that's failing or suffering performance problems. When a service is running slowly, it consumes resources on the client calling it. You want these microservice calls to fail fast so that the calling client can quickly respond and take appropriate action.
- *Fallback pattern*—When a service call fails, how you provide a "plug-in" mechanism that allows the service client to try to carry out its work through alternative means other than the microservice being called.
- *Bulkhead pattern*—Microservice applications use multiple distributed resources to carry out their work. This pattern refers to how you compartmentalize these calls so that the misbehavior of one service call doesn't negatively impact the rest of the application.

Figure 1.13 shows how these patterns protect the service consumer from being impacted when a service is misbehaving. These topics are covered in chapter 7.

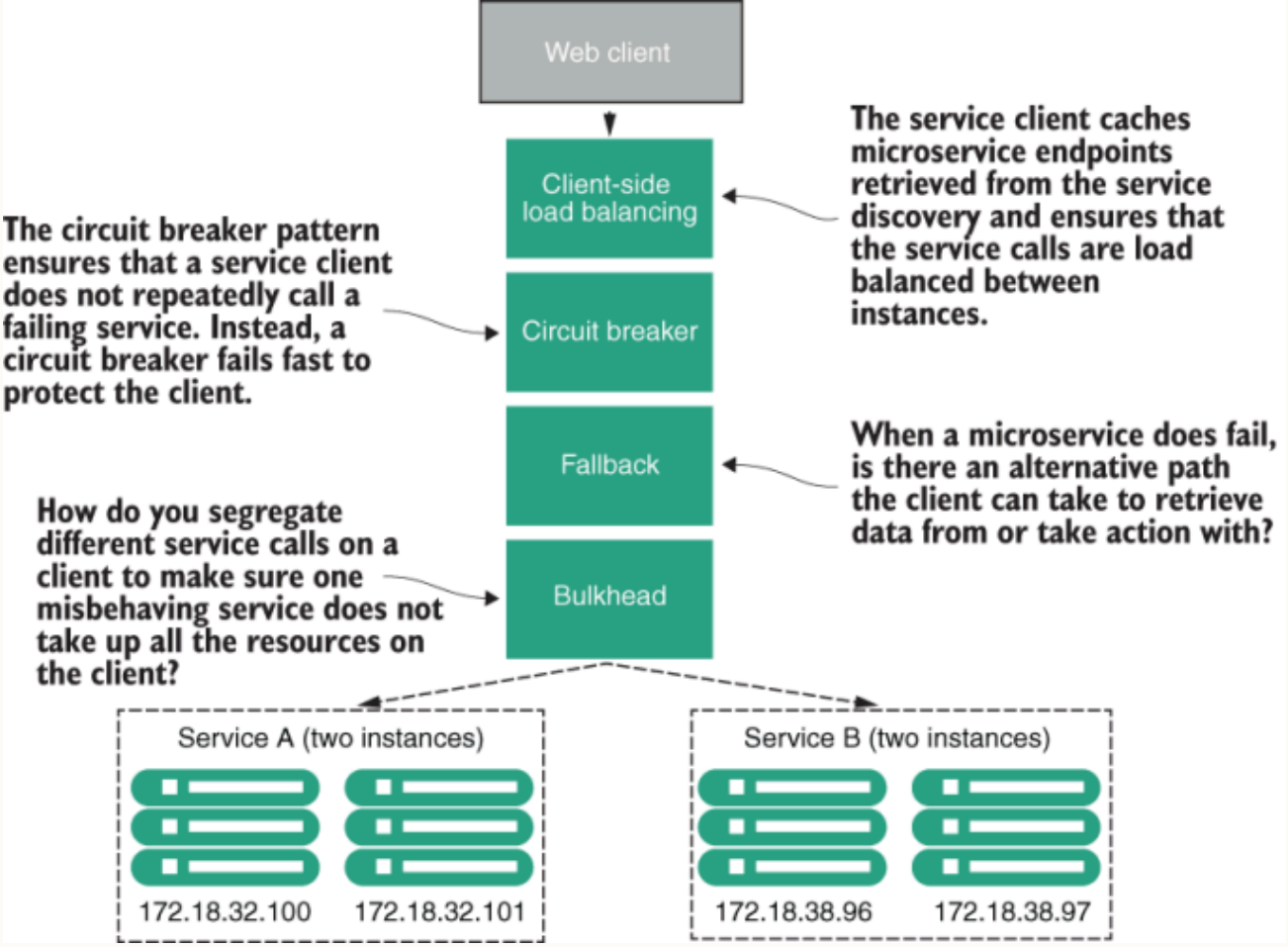


Figure 1.13 With microservices, you must protect the service caller from a poorly behaving service. Remember, a slow or down service can cause disruptions beyond the immediate service.

1.10 Microservice security patterns

To ensure that the microservices are not open to the public, it is important to apply the following security patterns to the architecture in order to ensure that only granted requests with proper credentials can invoke the services. Figure 1.14 shows how you can implement these three patterns to build an authentication service that can protect your microservices:

- *Authentication*—How you determine the service client calling the service is who they say they are.
- *Authorization*—How you determine whether the service client calling a microservice is allowed to undertake the action they're trying to take.
- *Credential management and propagation*—How you prevent a service client from constantly having to present their credentials for service calls involved in a transaction. To achieve this, we'll look at how you can use token-based security standards such as OAuth2 and JSON Web Tokens (JWT) to obtain a token that can be passed from service call to service call to authenticate and authorize the user.

What is OAuth 2.0?

OAuth2 is a token-based security framework that allows a user to authenticate themselves with a third-party authentication service. If the user successfully authenticates, they will be presented with a token that must be sent with every request.

The main goal behind OAuth2 is that when multiple services are called to fulfill a user's request, the user can be authenticated by each service without having to present their credentials to each service processing their request. While OAuth is covered in chapter 9, it's worthwhile to read the OAuth 2.0 documentation by Aaron Parecki (**https://www.oauth.com/**).

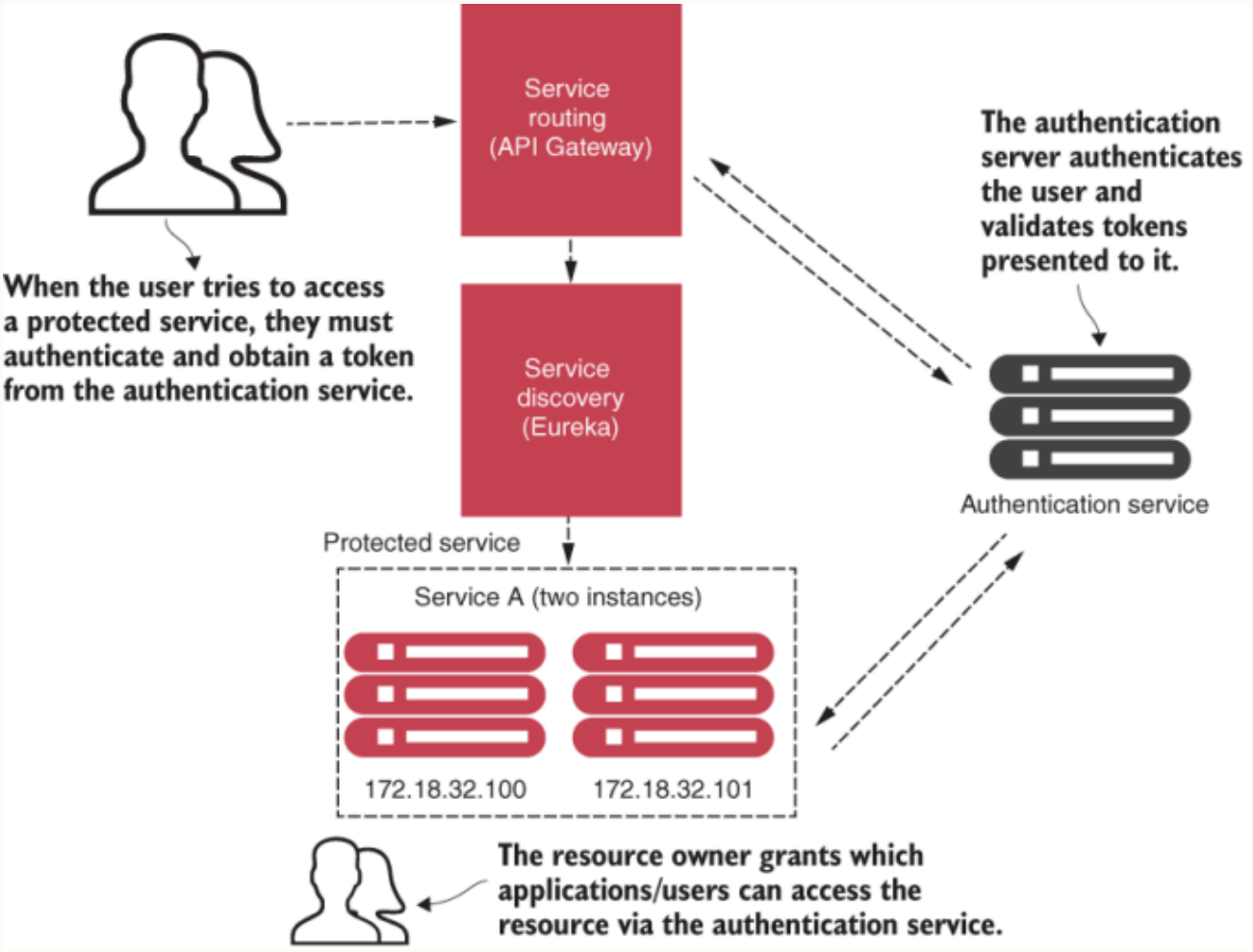


Figure 1.14 Using a token-based security scheme, you can implement service authentication and authorization without passing around client credentials.

1.11 Microservice logging and tracing patterns

The downside of a microservice architecture is that it's much more difficult to debug, trace, and monitor the issues because one simple action can trigger numerous microservice calls within your application. Further chapters will cover how to implement distributed tracing with Spring Cloud Sleuth, Zipkin, and the ELK Stack. For this reason, we'll

look at the following three core logging and tracing patterns to achieve distributed tracing:

- *Log correlation*—How you tie together all the logs produced between services for a single user transaction. With this pattern, we'll look at how to implement a correlation ID, which is a unique identifier that's carried across all service calls in a transaction and that can be used to tie together log entries produced from each service.
- *Log aggregation*—With this pattern, we'll look at how to pull together all of the logs produced by your microservices (and their individual instances) into a single queryable database across all the services involved and understand the performance characteristics of the services in the transaction.
- *Microservice tracing*—We'll explore how to visualize the flow of a client transaction across all the services involved and understand the performance characteristics of the transaction's services.

Figure 1.15 shows how these patterns fit together. We'll cover the logging and tracing patterns in greater detail in chapter 11.

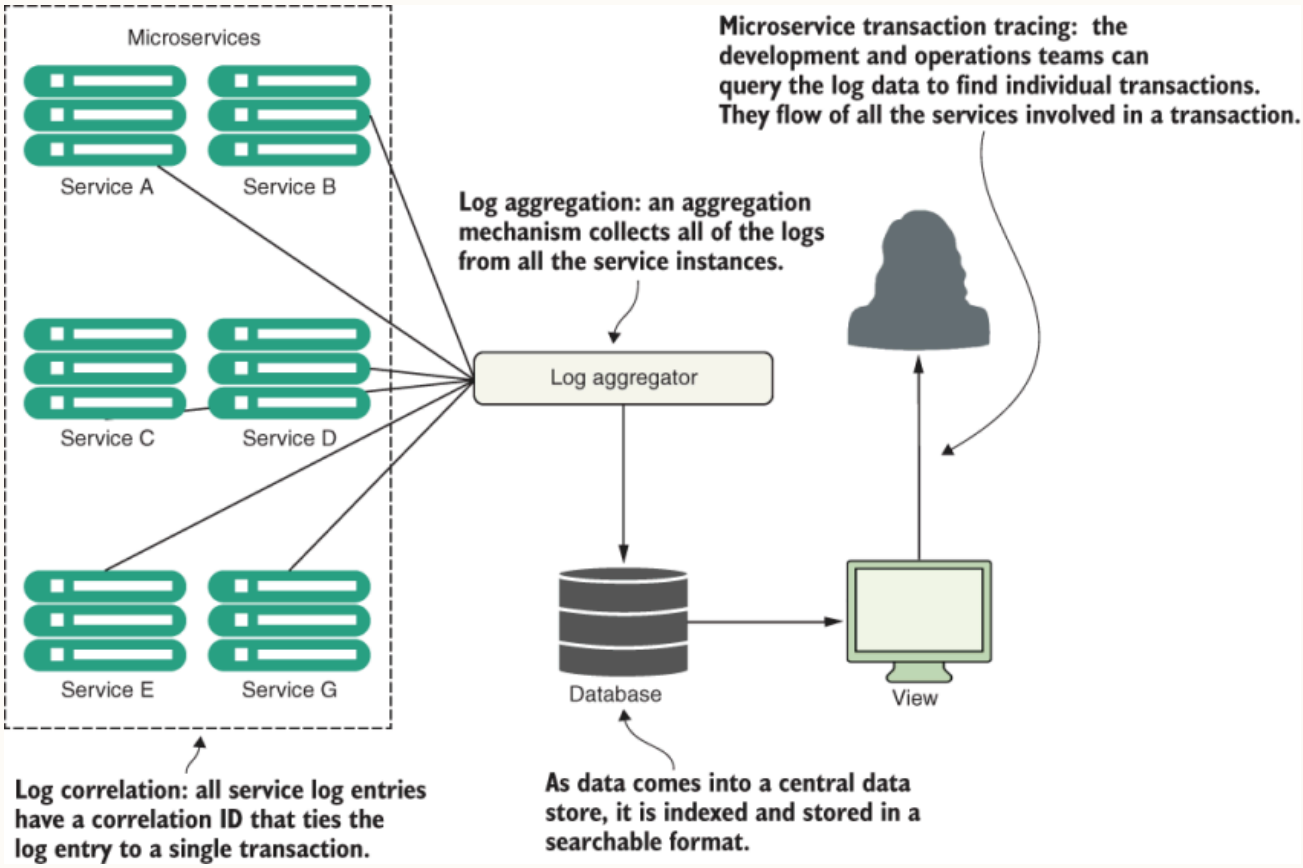


Figure 1.15 A well-thought-out logging and tracing strategy makes debugging transactions across multiple services manageable.

1.12 Application metrics pattern

The application metrics pattern deals with how the application is going to monitor metrics and warn of possible causes of failure within our applications. This pattern shows how the metrics service is responsible for getting (scraping), storing, and querying business-related data in order to prevent potential performance issues in our services. This pattern contains the following three main components:

- *Metrics*—How you create critical information about the health of your application and how to expose those metrics
- *Metrics service*—Where you can store and query the application metrics
- *Metrics visualization suite*—Where you can visualize business-related time data for the application and infrastructure

Figure 1.16 shows how the metrics generated by the microservices are highly dependent on the metrics service and the visualization suite. It would be useless to have metrics that generate and show infinite information if there is no way to understand and analyze that information. The metrics service can obtain the metrics using the pull or push style:

- With the push style, the service instance invokes a service API exposed by the metrics service in order to send the application data.
- With the pull style, the metrics service asks or queries a function to fetch the application data.

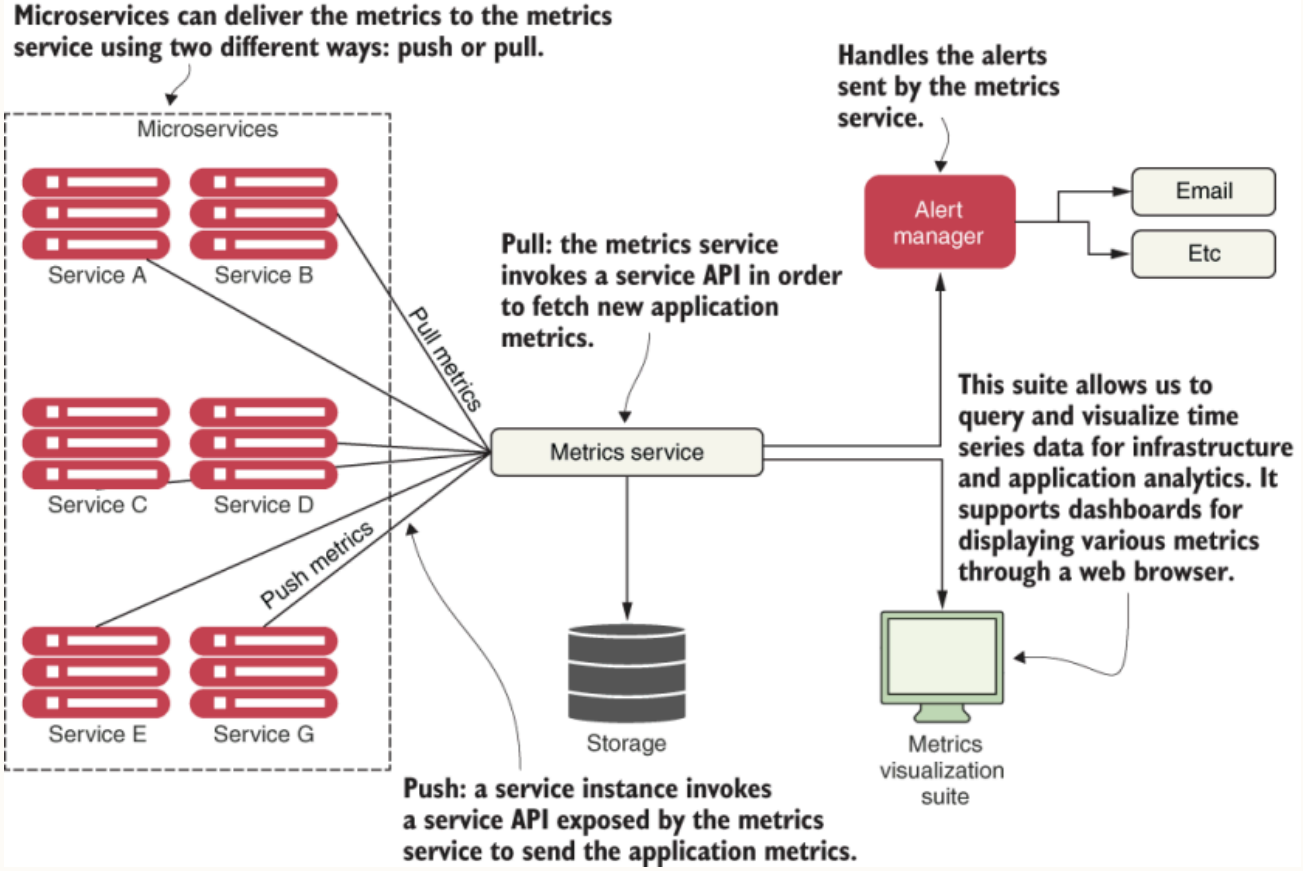


Figure 1.16 Metrics are pulled or pushed from the microservices and collected and stored in the metrics service to be shown using a metrics visualization suite and an alert management tool.

It's important to understand that monitoring metrics is an essential aspect of the microservices architectures and that the monitoring requirements in these kinds of architectures tend to be higher than monolithic structures due to their high distribution.

1.13 Microservice build/deployment patterns

One of the core parts of a microservice architecture is that each instance of a microservice should be identical to all its other instances. You can't allow *configuration drift* (something changes on a server after it's been deployed) to occur because this can introduce instability in your applications.

The goal with this pattern is to integrate the configuration of your infrastructure right into your build/deployment process so that you no longer deploy software artifacts such as Java WAR or EAR files to an already running piece of infrastructure. Instead, you want to build and compile your microservice and the virtual server image it's running on as part of the build process. Then, when your microservice gets deployed, the entire machine image with the server running on it gets deployed. Figure 1.17 illustrates this process. At the end of the book, we'll look at how to create your build/deployment pipeline. In chapter 12, we cover the following patterns and topics:

- *Build and deployment pipelines*—How you create a repeatable build and deployment process that emphasizes one-button builds and deployment to any environment in your organization.
- *Infrastructure as code*—How you treat the provisioning of your services as code that can be executed and managed under source control.
- *Immutable servers*—Once a microservice image is created, how you ensure that it's never changed after it has been deployed.
- *Phoenix servers*—How you ensure that servers that run individual containers get torn down on a regular basis and re-created from an immutable image. The longer a server is running, the more opportunity there is for configuration drift. A configuration drift can occur when ad hoc changes to a system configuration are unrecorded.

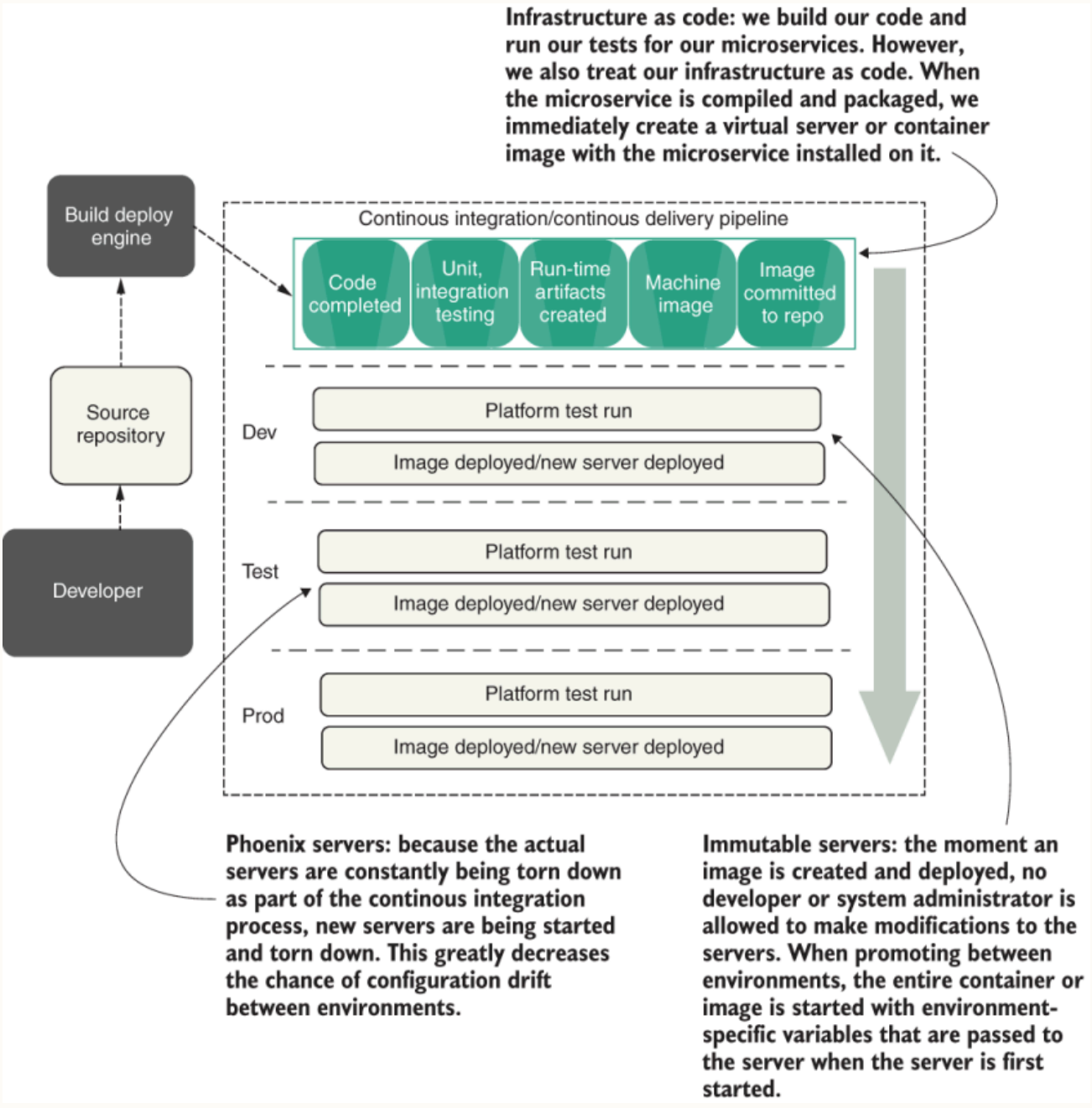


Figure 1.17 You want the deployment of the microservice and the server it's running on to be one atomic artifact that's deployed as a whole between environments.

Our goal with these patterns and topics is to ruthlessly expose and stamp out configuration drift as quickly as possible before it can hit your upper environments (stage or production).

NOTE For the code examples in this book (except in chapter 12), everything will run locally on your desktop machine. The first chapters can be run natively directly from the command line. Starting in chapter 3, all the code will be compiled and run as Docker containers.

Now that we've covered the patterns that we will use throughout the book, let's continue with the second chapter. In the next chapter, we'll cover the Spring Cloud technologies that we will use, some best practices for designing a cloud microservice-oriented application, and the first steps to creating our first microservice using Spring Boot and Java.

mmary

- Monolithic architectures have all processes tightly coupled, and these run as a single service.
- Microservices are extremely small pieces of functionality responsible for one specific area of scope.
- Spring Boot allows you to create both types of architectures.
- Monolithic architectures tend to be ideal for simple, lightweight applications, and microservices architectures are usually better for developing complex and evolving applications. In the end, selecting the software architecture depends entirely on your project size, time, and requirements, among other factors.
- Spring Boot simplifies the building of REST-based/JSON microservices. Its goal is to make it possible for you to build microservices quickly with nothing more than a few annotations.
- Writing microservices is easy, but fully operationalizing these for production requires additional forethought. There are several categories of microservice development patterns, including core development, routing patterns, client resiliency, security, application metrics, and build/deployment patterns.
- The microservice routing patterns deal with how a client application that wants to consume a microservice discovers the location of the service and is routed over to it.

- To prevent a problem in a service instance from cascading up and out to the consumers of the service, use the client resiliency patterns. These include the circuit breaker pattern to avoid making calls to a failing service, the fallback pattern to create alternative paths in order to retrieve data or execute a specific action when a service fails, the client load-balancing pattern to scale and remove all possible bottlenecks or points of failure scenarios, and the bulkhead pattern to limit the number of concurrent calls to a service in order to stop poor performance calls negatively affecting other services.
- OAuth 2.0 is the most common user authorization protocol and is an excellent choice for securing a microservice architecture.
- The build/deployment pattern allows you to integrate the configuration of your infrastructure right into your build/deployment process so that you no longer deploy software artifacts such as Java WAR or EAR files to an already running piece of infrastructure.

Previous chapter
front matter

Next chapter
2 Exploring the microservices world with Spring Cloud